

Utilizando *um dicionário*

O **dicionário** guarda pares **chave: valor**: em vez de "o item da posição 2", você pede "o valor da chave idade". É a estrutura mais usada do Python para representar registros, configurações, contagens e dados vindos de APIs. Nesta aula: criar, ler, alterar, o `KeyError` e o `.get()`, percorrer, contar e dicionários dentro de dicionários.

`{"chave": valor}`

`d["chave"]`

`.get()`

`KeyError`

`.keys()` · `.values()` · `.items()`

Contagem

Aninhados

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: Dictionaries; Built-in Types: Mapping Types), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Pares *chave: valor*

criar.py

```
aluno = {"nome": "Ana", "idade": 29, "cidade": "Recife"}  
print(aluno["nome"])      # acesso pela chave  
print(len(aluno))         # quantos pares chave: valor  
print("idade" in aluno)   # in procura nas CHAVES  
print("Ana" in aluno)     # False: "Ana" é valor, não chave
```

SAÍDA

```
Ana  
3  
True  
False
```

- ✓ Chaves `{ }`, pares separados por vírgula, `chave: valor`
- ✓ Acesso pela **chave**: `aluno["nome"]`
- ✓ Chaves costumam ser textos; podem ser números ou tuplas (aula 10), nunca listas
- ✓ Chaves são **únicas**; valores podem repetir e ser de qualquer tipo
- ✓ `in` procura nas **chaves**

Desde o Python 3.7, o dicionário guarda a **ordem** em que as chaves foram inseridas.

Incluir, atualizar *e remover*

alterar.py

```
aluno = {"nome": "Ana", "idade": 29}
aluno["idade"] = 30          # chave existe: atualiza
aluno["email"] = "ana@ex.com" # chave nova: adiciona
print(aluno)
removido = aluno.pop("email") # remove e devolve o valor
del aluno["idade"]           # remove sem devolver
print(removido, aluno)
```

SAÍDA

```
{'nome': 'Ana', 'idade': 30, 'email': 'ana@ex.com'}
ana@ex.com {'nome': 'Ana'}
```

- ✓ `d[chave] = valor` : atualiza se a chave existe, **cria** se não existe
- ✓ `.pop(chave)` remove e devolve o valor
- ✓ `del d[chave]` remove sem devolver
- ✓ `.update(outro)` ou `|` juntam dicionários (seção 06)

KeyError e o .get()

erro_chave.py

```
aluno = {"nome": "Ana"}  
print(aluno["email"])
```

```
Traceback (most recent call last):  
  File "erro_chave.py", line 2, in <module>  
    print(aluno["email"])  
          ~~~~~^~~~~~  
KeyError: 'email'
```

Pedir com `[ ]` uma chave que não existe dá **KeyError**.

get.py

```
aluno = {"nome": "Ana", "idade": 29}  
print(aluno.get("email"))           # None: não existe  
print(aluno.get("email", "sem email")) # valor padrão  
print(aluno.get("nome", "?"))
```

```
None  
sem email  
Ana
```

`.get(chave, padrão)` devolve o valor ou o padrão, sem erro.
Recomendação do guia (cap. 9): use `.get()` quando a chave pode faltar.

keys, values *e items*

percorrer.py

```
estoque = {"maçã": 12, "banana": 30, "uva": 0}
print(list(estoque.keys()))
print(list(estoque.values()))
for fruta, qtd in estoque.items():
    print(f"{fruta:<8}{qtd:>4}")
print("Total:", sum(estoque.values()))
```

SAÍDA

```
['maçã', 'banana', 'uva']
[12, 30, 0]
maçã      12
banana    30
uva        0
Total: 42
```

MÉTODO

DEVOLVE

.keys()

As chaves

.values()

Os valores

.items()

Pares (chave, valor) como tuplas

✓ `for chave, valor in d.items()` desempacota cada par (aula 10)

✓ `for chave in d` percorre só as chaves

✓ O `for` em detalhe é a aula 14

Contando *com um dicionário*

contar.py · QUANTAS PALAVRAS COMEÇAM COM CADA LETRA

```
frase = "o rato roeu a roupa do rei de roma"
contagem = {}
for palavra in frase.split():
    letra = palavra[0]
    contagem[letra] = contagem.get(letra, 0) + 1
print(contagem)
```

SAÍDA

```
{'o': 1, 'r': 5, 'a': 1, 'd': 2}
```

- ✓ `contagem.get(letra, 0) + 1`: se a letra ainda não apareceu, começa do 0
- ✓ Um dos padrões mais usados do Python: contar votos, palavras, vendas por produto

O módulo `collections` tem o `Counter`, que faz essa contagem pronta: `Counter(p[0] for p in frase.split())`. Vale conhecer depois de entender o padrão manual.

Dicionários *dentro de dicionários*

aninhado.py

```
turma = {
    "ana": {"nota": 9.0, "faltas": 1},
    "bia": {"nota": 7.5, "faltas": 4},
}
print(turma["bia"]["nota"])
turma["caio"] = {"nota": 8.0, "faltas": 0}
print(sorted(turma))      # sorted de um dict ordena as chaves
config = {"tema": "claro", "idioma": "pt"} | {"tema": "escuro"}
print(config)             # | junta; a direita vence
```

SAÍDA

```
7.5
['ana', 'bia', 'caio']
{'tema': 'escuro', 'idioma': 'pt'}
```

- ✓ Valor pode ser outro dicionário (ou lista): é como o JSON das APIs
- ✓ Duas chaves em sequência: `turma["bia"]["nota"]`
- ✓ `sorted(dicionario)` devolve as chaves em ordem
- ✓ `a | b` junta (Python 3.9+); se a chave se repete, vale a da direita

Lembra das listas aninhadas da aula 09? Aqui os campos têm **nome**, e o código fica bem mais legível: `["nota"]` em vez de `[1]`.

Uma *agenda de telefones*

Enunciado: crie uma agenda com dois contatos, inclua um terceiro, peça um nome para buscar (sem ligar para maiúsculas) e mostre o telefone ou "Não encontrado". No fim, liste os contatos em ordem.

Uma resposta possível

```
# Exercício: agenda de telefones
agenda = {"Ana": "81 99999-0001", "Bia": "11 98888-0002"}
agenda["Caio"] = "21 97777-0003"
nome = input("Buscar: ").strip().title()
print(agenda.get(nome, "Não encontrado"))
print(len(agenda), "contatos:", ", ", ".join(sorted(agenda)))
```

SAÍDA (DIGITOU "BIA")

```
Buscar: bia
11 98888-0002
3 contatos: Ana, Bia, Caio
```

- ✓ `.title()` transforma "bia" em "Bia", igual à chave
- ✓ `.get()` com padrão evita o KeyError
- ✓ Desafio: guarde e-mail e telefone para cada contato

Teste o que *ficou*

1. `d = {"a": 1}; d["b"] = 2`. O que acontece?

A) `KeyError`

B) Cria a chave "b"

C) Troca "a" por "b"

D) Nada

B. Atribuir cria ou atualiza.

2. `d.get("x", 0)` quando "x" não existe:

A) `KeyError`

B) `None`

C) 0

D) "x"

C. O padrão informado.

3. `"Ana" in {"nome": "Ana"}` dá:

A) `True`

B) `False`

C) Erro

D) "nome"

B. in procura nas chaves.

4. Para percorrer chave e valor juntos:

A) `d.keys()`

B) `d.values()`

C) `d.items()`

D) `d.pairs()`

C. `for k, v in d.items()`.

Dados com *nome*

O dicionário guarda pares **chave: valor**, acessados pela chave. Atribuir cria ou atualiza; `[` com chave inexistente dá **KeyError**, e `.get()` evita o erro. `.items()` percorre os pares, o padrão de contagem resolve muitos problemas e dicionários aninhados representam dados como JSON.

`d["chave"]`

`.get(chave, padrão)`

`.items()`

Contagem

a | b

Próxima aula → 13 · Trabalhando com Condições

Fontes: docs.python.org/3/tutorial/datastructures.html#dictionaries; docs.python.org/3/library/stdtypes.html#mapping-types-dict; docs.python.org/3/library/collections.html#collections.Counter, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.